

Kora Kitchen

HTML, CSS, and JavaScript Code Summary

A plain-language guide to the restaurant website in `index.html`, `styles.css`, and `script.js`.

[Restaurant website](#)[Responsive layout](#)[Shopping cart](#)[Interactive UI](#)

1. Project Overview

Kora Kitchen is a single-page restaurant ordering website. The page combines a fixed navigation bar, a food-focused hero section, category cards, popular dish cards, a cart sidebar, an image gallery, restaurant information, newsletter signup, and a footer.

The three files work together like this:

File	Responsibility
<code>index.html</code>	Defines the content, semantic sections, buttons, images, links, forms, and data attributes.
<code>styles.css</code>	Controls layout, colors, typography, cards, spacing, hover states, animation, and responsive breakpoints.
<code>script.js</code>	Adds browser behavior after <code>DOMContentLoaded</code> : menus, favorites, quantities, cart updates, search, newsletter validation, and scrolling.

2. HTML Summary

Document setup

- Uses HTML5 with `<!DOCTYPE html>` and `lang="en"`.
- Includes responsive viewport metadata.
- Loads Font Awesome from a CDN for navigation, food, cart, social, and utility icons.
- Loads `styles.css` for appearance and `script.js` with `defer` so JavaScript runs after the document is parsed.

Header and navigation

The fixed header contains the Kora Kitchen logo, links to Home, Menu, About, and Contact, a food search box, user icon, cart icon with a numeric badge, and a mobile menu button with the ID `menuButton`.

Hero section

The Home section introduces the restaurant with the headline “Your Cravings, Our Kitchen!”, supporting copy, an Order Now anchor, a View Menu button, and three selling points: Fast Delivery, Best Quality, and 24/7 Support.

Category section

The Categories section contains six clickable category cards: Burgers, Pizza, Chicken, Drinks, Desserts, and Salads. Each card uses a `data-category` value and an image.

Popular dishes and cart

The Menu section displays four dish cards: Cheese Burger, Chicken Pizza, Fried Chicken, and Creamy Pasta. Every card includes an image, favorite button, title, five-star rating, price, quantity control, and Add to Cart button.

The cart sidebar begins with three sample cart items. Each item has a name, price, quantity controls, and total. The sidebar also contains subtotal, delivery fee, total, and Checkout button.

Supporting sections

- A marquee message and animated meal image gallery.
- An About Us section with restaurant image, description, and Learn More button.
- A Why Choose Us grid with four feature cards.
- A newsletter form with an email input and Subscribe button.
- A footer with branding, social links, quick links, menu links, help links, contact details, and copyright text.

3. CSS Summary

Global foundation

The universal selector resets padding and margin, applies `box-sizing: border-box`, and uses Arial as the main font. The body uses a white background and dark text.

Navigation styling

The header is fixed at the top with a black background and high z-index. The navigation uses flexbox. Links are white and change to orange with an underline on hover. The cart badge is a red circular counter positioned over the cart icon.

At widths below 800px, the menu button appears, navigation links become a hidden vertical dropdown, and the search box becomes narrower.

Hero and categories

The hero uses a dark overlay gradient over `images/hero-burger.jpeg`, cover sizing, large heading text, orange accent text, and flex alignment. Category cards use a six-column grid on large screens, three columns below 900px, and two columns on smaller screens. Hovering lifts cards and enlarges their images; the active category receives an orange border and pale orange background.

Dish cards and quantity controls

Popular dishes use a two-column grid and white cards with rounded corners, shadows, image cropping, rating colors, orange prices, and aligned card actions. The quantity control is a bordered flex row containing minus, quantity, and plus buttons. Add to Cart buttons use the orange brand color and darken on hover.

Cart sidebar

The sidebar is a white, shadowed panel that becomes sticky on larger screens. Cart rows use flex layout. Quantity buttons are small bordered controls. Summary rows align labels and values using flexbox. The checkout button spans the full sidebar width.

Gallery, content sections, and footer

The gallery has a dark marquee strip and a horizontally animated image row using the `scrollImages` keyframes. About, feature, newsletter, and footer areas use consistent spacing, orange accents, dark backgrounds where appropriate, grid layouts, and hover states.

Responsive behavior

At 992px and below, the dish/cart layout becomes one column, feature cards become two columns, the footer becomes two columns, and the About section becomes one column. At 576px and below, dishes and feature cards become one column. Additional 600px and 400px breakpoints adjust category cards and navigation sizing.

4. JavaScript Summary

All behavior is placed inside a `DOMContentLoaded` callback, which prevents the code from running before the HTML elements are available.

Category selection

Each `.category-card` receives a click listener. Clicking one removes `active` from all category cards, adds it to the clicked card, and logs its `data-category` value.

Mobile navigation

The menu button toggles the `open` class on navigation links and the `active` class on the menu button. Clicking a navigation link closes the mobile menu.

Favorites

Heart buttons locate their dish card and read the dish name from its `h3`. Favorite names are stored in `localStorage` under `favorites`. On page load, saved favorites restore their active state. Clicking a heart toggles the icon between regular and solid styles and adds or removes the dish name from storage.

Dish-card quantities

Each `.qty-control` gets plus and minus listeners. Plus increases the displayed quantity. Minus decreases it but stops at 1. The Add to Cart handler reads this quantity when creating or updating a cart item.

Add to Cart

When Add to Cart is clicked, the script finds the card, extracts its name, price, and quantity, and searches the cart for an existing matching item. Existing items receive the added quantity. New items are created dynamically with name, price, quantity controls, and calculated total. The button temporarily changes to a green “Added” state.

Cart quantities and totals

The cart uses delegated click handling on `.cart-sidebar`. This is important because newly added cart items do not exist when the page first loads. The handler identifies the clicked cart quantity button, changes its row quantity, and calls `updateCartSummary()`.

`updateCartSummary()` recalculates every item total, subtotal, delivery fee, grand total, cart header count, and top navigation cart badge. Delivery is 1,500 when the cart has items and 0 when it is empty.

Other interactions

- Clicking the cart icon scrolls to the cart and briefly highlights it.
- The trash icon removes all cart items and recalculates totals.
- Checkout shows an empty-cart alert or a thank-you message with the total.
- View Menu smoothly scrolls to the menu section.
- Learn More inserts and removes additional About text.
- The newsletter form validates an email value, shows a confirmation alert, and resets.
- Search filters dish cards live as the user types and also works with Enter or the search button.

5. Current Implementation Notes

Working fix: Cart quantity controls use event delegation, so quantities on dynamically added dishes work correctly.

- The Font Awesome icons depend on an internet connection because they load from a CDN.
- The category click handler currently selects and logs categories; it does not filter the dishes by category.
- The cart starts with hard-coded sample items and does not persist cart data in localStorage.
- The HTML contains a few minor cleanup opportunities, such as an empty `id` attribute on the Order Now link and a list item used inside the title-line decoration.
- The CSS references `--dark-orange` without defining that variable, so the hero button hover may fall back to the browser's inherited behavior.

Prepared from the current Kora Kitchen project files on September 7, 2026.